



**ISTANBUL UNIVERSITY-CERRAHPASA
FACULTY OF ENGINEERING,
DEPARTMENT OF COMPUTER ENGINEERING
FILE ORGANIZATION PROJECT**

REPORT DELIVERY DATE: 15.06.2026

PREPARER :

<u>Student Number</u>	<u>Name</u>	<u>Surname</u>
1306230084	Rabianur	Becit

Associate: Assoc. Prof. DERYA YILTAŞ KAPLAN

Assistant: Res. Asst. MELİKE BAŞER GÜVEN

1. INTRODUCTION

Purpose of the Project:

The objective of this project is to clean and standardize a large-scale password dataset and subsequently store it using three distinct methods: Plain Text (Raw), Hashed (MD5), and Compressed (zlib). The core aim is to comparatively analyze the effects of these storage representations on disk usage, search performance, and system resource consumption (CPU and RAM).

Importance of the Project:

Password lists are widely used in database security and authentication processes; therefore, selecting the appropriate storage architecture is a critical step for system performance. This study provides concrete measurements demonstrating how different representational forms of the same dataset affect system resources and query speed.

2. TOOLS AND TECHNOLOGIES

- **Programming Language:** C
- **Libraries:**

Standard Libraries: `<stdio.h>`, `<stdlib.h>`, `<string.h>` (Used for file I/O operations, dynamic memory allocation, and string manipulations)

System APIs (Windows):

<windows.h>: Used for high-precision nanosecond timing (QueryPerformanceCounter), CPU time extraction (GetProcessTimes), and fast disk size queries (GetFileSizeEx).

<psapi.h>: Used for accurately monitoring the process's active RAM footprint (GetProcessMemoryInfo - WorkingSetSize).

Third-Party / External Libraries:

md5: A self-contained, lightweight C implementation of the MD5 Message-Digest Algorithm based on RFC 1321. Used exclusively for generating 32-character hashes in the Hashed Storage method.

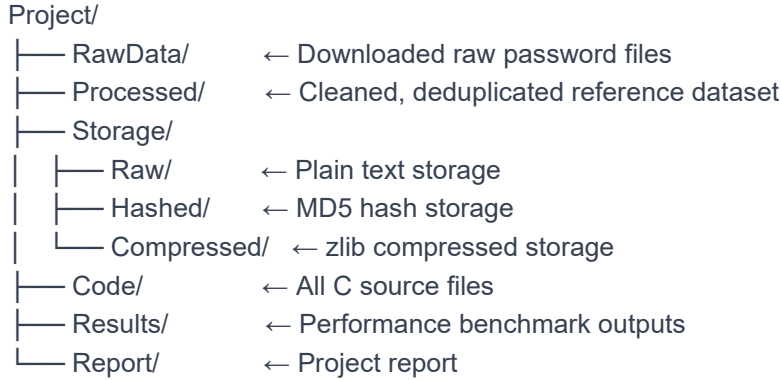
miniz (v3.0.2): A single-file, high-performance, zlib-compatible lossless data compression library. Integrated without external dependencies to handle the deflation and inflation of the dataset for the Compressed Storage method.

- **Operating System:** Windows 11
- **Development Environment:** Visual Studio Code – MinGW/GCC Compiler
- **Measurement Tools:** QueryPerformanceCounter (nanosecond-precision timing), GetProcessMemoryInfo (RAM), GetProcessTimes (CPU)
- **Dataset:** Daniel Miessler – SecLists, Common-Credentials folder

3. PROJECT DESIGN

3.1 Folder Structure

The project is organized according to the following directory hierarchy:



3.2 System Data Flow

The execution of the program follows a strict, three-phase linear pipeline:

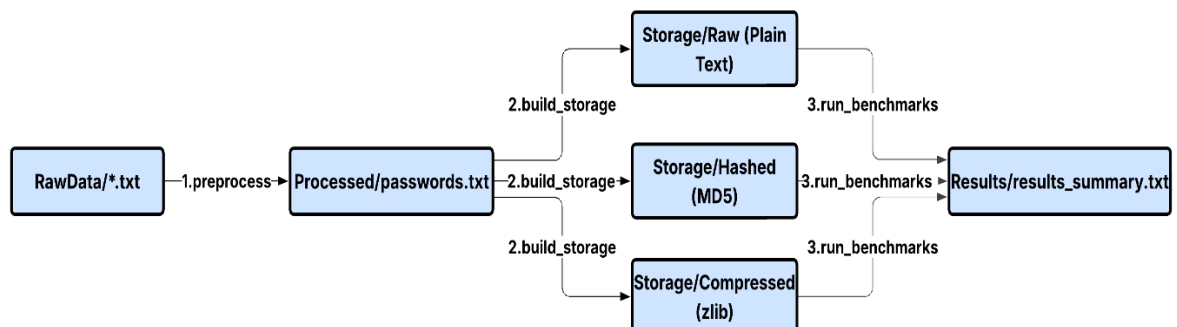
1. Preprocessing Phase: The system first invokes the `preprocess()` function, which scans all raw text files located in the `RawData/` directory. It merges the data, eliminates empty lines, and removes duplicate entries to generate a single, unified reference dataset at `Processed/passwords.txt`.

2. Storage Generation Phase: Next, the `build_storage()` function reads this unique reference dataset and transforms it into three distinct structural formats: a baseline plain text copy (Raw), an MD5 hashed and sorted format (Hashed), and a zlib-deflated binary format (Compressed). These are saved into their respective subdirectories within `Storage/`.

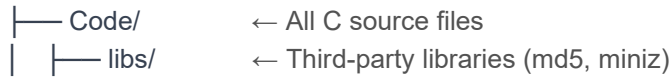
3. Benchmarking Phase: Finally, the `run_benchmarks()` function selects 100 random queries from the reference dataset and evaluates the search speed, CPU overhead, and RAM consumption across the three storage methods. The final performance metrics and automated evaluations are exported directly to `Results/results_summary.txt`.

The overall data flow of the system is as follows:

RawData/*.txt → preprocess() → Processed/passwords.txt → build_storage() → Storage/{Raw, Hashed, Compressed} → run_benchmarks() → Results/results_summary.txt



3.3 Code Modules



- **main.c:** Entry point of the program; orchestrates all modules in sequence.
- **preprocess.c:** Reads raw data files, removes empty lines and duplicates, writes the cleaned output to Processed/.
- **storage.c:** Converts the processed dataset into Raw, Hashed, and Compressed storage formats.
- **search.c:** Manages search algorithms and benchmark processes across all storage structures.
- **compression.c:** Performs LZSS deflation and inflation operations using the miniz library.
- **utils.c:** Handles CPU, RAM, and timing measurements via Windows API calls.
- **libs/md5.c:** Self-contained MD5 hash generator with no external dependencies (RFC 1321).
- **libs/miniz.c:** A single-file, zlib-compatible lossless data compression library used for the Compressed storage method.

Note on Header Files: With the exception of main.c, every C module in this project is accompanied by a corresponding header file (.h). These header files cleanly define function prototypes, struct definitions, and necessary macros, ensuring strict modularity and interface separation across the entire codebase.

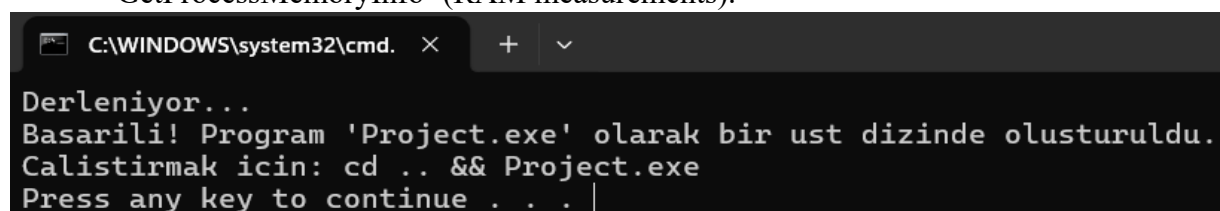
3.4 Compilation and Build Process

The project does not rely on heavy IDE-specific build systems; instead, it is compiled directly using the GCC (MinGW) compiler via a provided batch script (`compile.bat`). To ensure maximum execution speed during the benchmarking phase, the `-O3` compiler optimization flag was utilized.

The exact compilation command used in the project is:

```
`gcc -O3 -Wall main.c preprocess.c storage.c search.c compression.c utils.c libs/md5.c  
libs/miniz.c -o ..\Project.exe -lpsapi`
```

- **-O3:** Instructs the compiler to apply maximum performance optimizations.
- **-Wall:** Enables all compiler warnings to ensure code safety and cleanliness.
- **-lpsapi:** Links the Windows Process Status API library required for `GetProcessMemoryInfo` (RAM measurements).



```
C:\WINDOWS\system32\cmd.  ×  +  v  
Derleniyor...  
Basarili! Program 'Project.exe' olarak bir ust dizinde olusturuldu.  
Calistirmak icin: cd .. && Project.exe  
Press any key to continue . . . |
```

Figure 1: Successful execution of the compile.bat script, compiling the project without any warnings or errors.

4. DATA PREPROCESSING

The preprocess.c module reads all text files in the RawData directory sequentially. Empty lines and trailing whitespace are stripped; records are loaded into a dynamically allocated string array. No case normalization was applied - all passwords are preserved in their original form.

The data is sorted in memory using qsort, and consecutive duplicate entries are eliminated by scanning the sorted array. The remaining unique passwords are written to Processed/passwords.txt. All subsequent search and conversion operations operate exclusively on this file; no component under Storage reads directly from RawData.

As part of the structural preparation and preprocessing strategy, the mechanism for hash generation was also established. Each standardized plaintext password is mathematically transformed using a self-contained MD5 (Message-Digest Algorithm 5) implementation based on the RFC 1321 standard. This algorithm takes the variable-length strings and generates a fixed-length 32-character hexadecimal digest. Although the actual writing of these hashes occurs in the subsequent storage phase, establishing this MD5 standardization ensures strict data uniformity and enables the efficient lexicographical sorting required later in the pipeline.

5. STORAGE METHODS

5.1 Raw Storage

The Processed/passwords.txt file is copied as-is to the Storage/Raw/ directory in plain-text format, one password per line. No transformation or compression is applied. This method serves as the performance baseline against which the other two methods are evaluated.

5.2 Hashed Storage

The 32-character hexadecimal MD5 digest of each password is computed and stored in place of the plaintext. The hashes are sorted lexicographically during file creation to enable efficient binary search. The resulting file is saved to Storage/Hashed/passwords_md5.txt. No plaintext password is retained in this file, providing one-way (irreversible) storage.

5.3 Compressed Storage

The entire processed dataset is read into memory and compressed into a single binary block using the zlib deflation algorithm via the miniz library. The output is saved to Storage/Compressed/passwords.zlib. The original (uncompressed) file size is prepended to the binary file so that the correct amount of memory can be allocated (malloc) during decompression.

6. SEARCH AND PERFORMANCE MEASUREMENTS

6.1 Search Algorithms

Search benchmarks were conducted using 100 passwords randomly selected from the Processed/ dataset. Each password was searched across all three storage structures.

- **Raw Search – Linear Search $O(N)$:** The passwords.txt file is opened for each query and read line by line with fgets until the target string is found or EOF is reached. Memory consumption is minimal, but search time is the longest among the three methods.
- **Hashed Search – Binary Search $O(\log N)$:** Because all MD5 hashes have a fixed record length (34 bytes: 32 characters + \r\n) and the file is pre-sorted, fseek is used to perform direct on-disk binary search without loading the entire file into RAM. This minimizes memory usage significantly.
- **Compressed Search:** The zlib binary file is fully decompressed into RAM. An in-memory linear search is then performed using strstr on the decompressed buffer. Memory usage and decompression overhead are high, but the on-disk footprint is the smallest of all three methods.

6.2 Measurement Methodology

- **Time:** QueryPerformanceCounter was used for nanosecond-precision timing; results are reported in microseconds (μs). Average and maximum search times were computed over 100 queries.
- **CPU:** User-mode and kernel-mode cycle deltas were collected via GetProcessTimes and expressed as a percentage (%) of total CPU utilization during search operations.
- **RAM:** WorkingSetSize was monitored through GetProcessMemoryInfo and recorded in megabytes (MB) for each storage method.
- **Disk Size:** The total size of the output files in each storage directory was measured dynamically via the GetFileSizeEx Windows API and formatted into bytes/KB/MB.

7. RESULTS AND EVALUATION

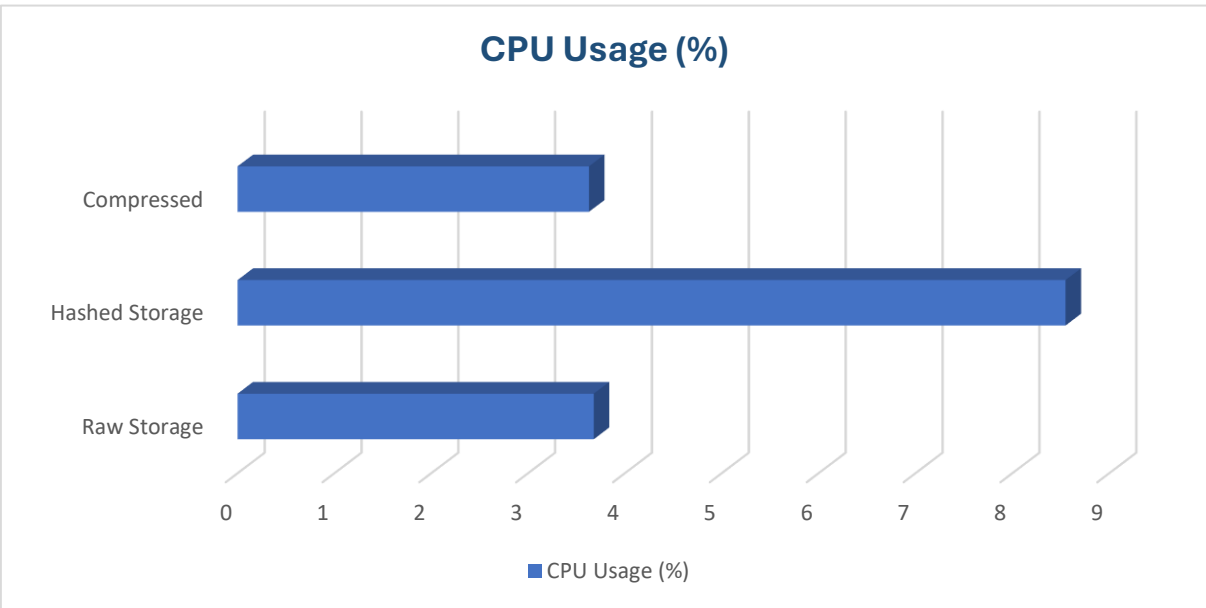
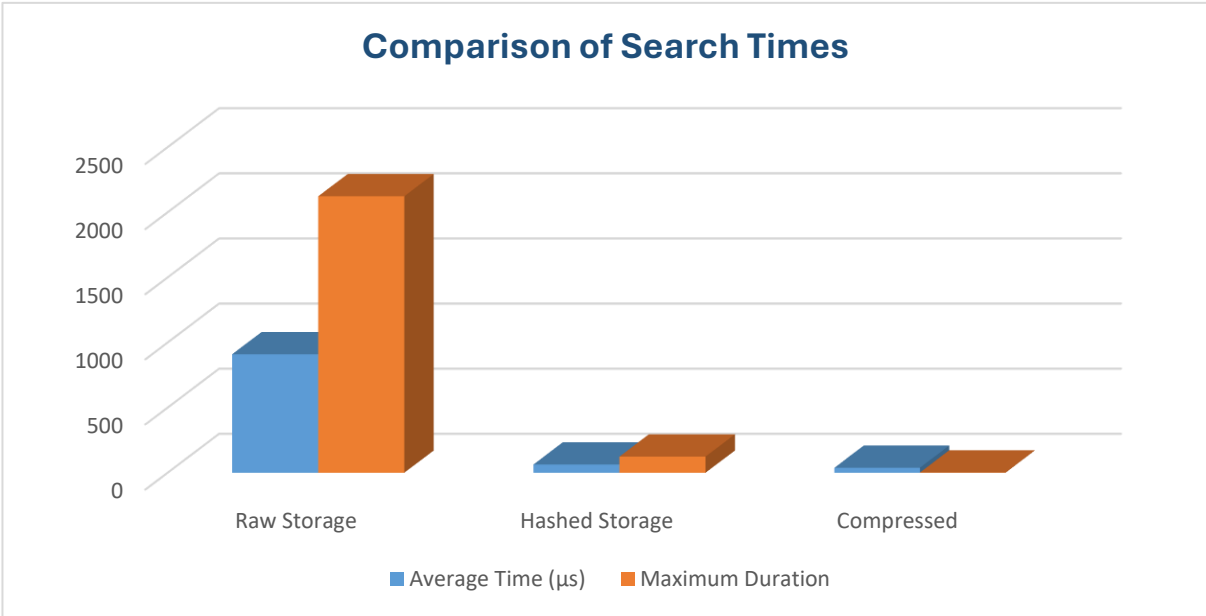
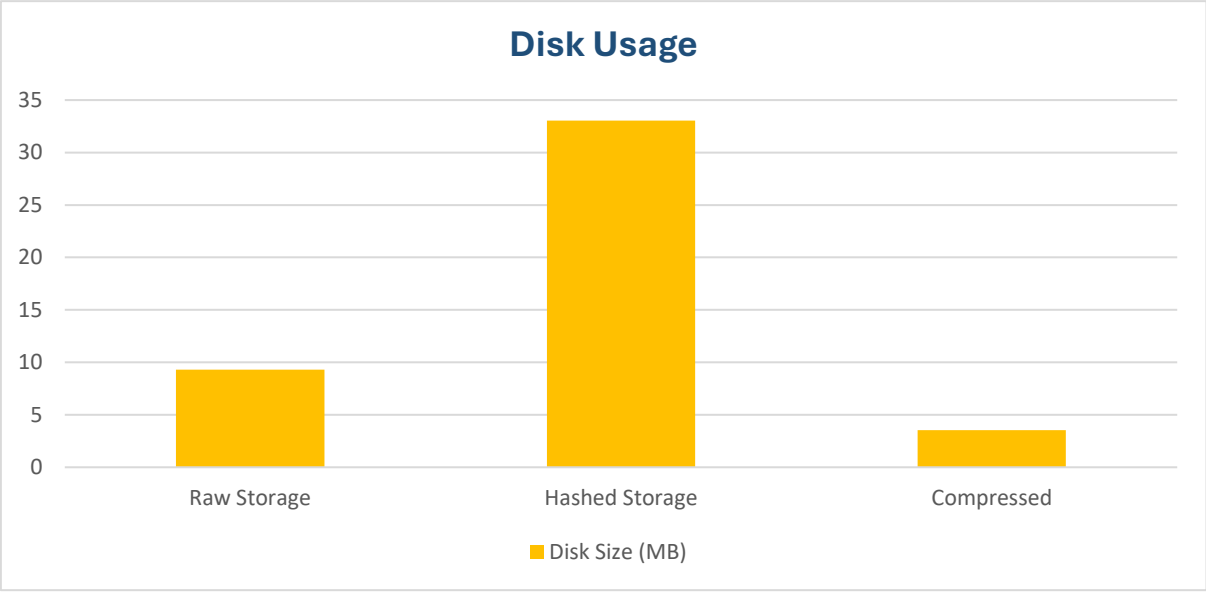
7.1 Performance Summary Table and Graphs

The table below must be filled in with the actual values from Results/results_summary.txt after running the benchmarks.

Method	Disk Size	Avg Search (us)	Std Dev (us)	Max Search (us)	CPU Usage	RAM Usage
Raw Storage	9.30 MB	910.02	525.91	2123.10	%3.68	4.82 MB
Hashed Storage	33.04 MB	63.21	12.83	123.50	%8.55	4.82 MB
Compressed	3.53 MB	38.12	21.40	107.70	%3.63	4.84 MB

ADDITIONAL METRICS:

- Compression Ratio (Compressed / Raw): 0.3797
- Decompression Time: 41.94 ms



7.2 Discussion and Comparison

EVALUATION AND COMPARISON:

1. **SPEED (Search Time):** Hashed Storage provides the fastest direct lookup (avg 63 us) because it uses Binary Search without loading the whole file. While Compressed Storage might seem faster (avg 38 us), this is only because it searches entirely in RAM *after* a massive 41.94 ms decompression penalty. Raw Storage is the slowest method due to its reliance on linear disk reading.
2. **DISK USAGE:** Compressed Storage occupies the least disk space (3.53 MB) thanks to the zlib algorithm. Hashed Storage is the most disadvantageous in terms of disk space (33.04 MB) due to the 32-character length of MD5 hashes.
3. **RAM AND CPU:** Since the Compressed method requires decompression into memory before searching (41.94 ms), it incurs additional time and RAM overhead, especially during the first run. While Raw storage reads directly from the disk, Hashed storage requires instantaneous CPU processing power to calculate the MD5 hash before searching.

- **Raw Storage:** Requires no transformation cost, but yields the slowest search times. Disk I/O becomes a bottleneck for large datasets. Suitable for small datasets or infrequently queried systems.
- **Hashed Storage:** Provides one-way storage, making it ideal for security-sensitive use cases. On-disk binary search via fseek minimizes RAM usage. Best suited for authentication and password verification systems.
- **Compressed Storage:** Significantly reduces disk footprint (Compression Ratio < 1). However, the entire file must be decompressed into RAM before any search can be performed, introducing latency and high memory overhead. Well suited for archival purposes; impractical for real-time query systems.

7.2.1 Which Method is Better for Which Scenario?

Based on the performance metrics and architectural behaviors observed during the benchmarks, there is no single "best" method; instead, each method excels in a specific domain:

- **Best for Speed and Security (Authentication): Hashed Storage** is unequivocally the best method for systems requiring rapid query responses and high security (due to one-way MD5 hashing). It is the ideal choice for modern web applications and login verification systems.
- **Best for Archival and Disk Space: Compressed Storage** is the superior method when disk capacity is the primary constraint. It heavily reduces the physical footprint, making it the best choice for cold storage, backups, or offline data archiving where immediate query speed is not a priority.
- **Best for Small or Simple Systems: Raw Storage** is the best option only for very small datasets or simple applications where development speed is more important than hardware efficiency, as it requires zero transformation overhead.

7.2.2 Advantages and Disadvantages of Each Method

Method	Advantages	Disadvantages
Raw Storage	Requires absolutely no transformation, compression, or hashing cost during data ingestion. Data is easily readable by humans.	Yields the slowest search times (Linear Search) and disk I/O rapidly becomes a massive bottleneck as the dataset grows. It also provides zero data security.
Hashed Storage (MD5)	Provides irreversible security, ensuring plain passwords are not exposed. Utilizing on-disk binary search via fseek minimizes RAM usage to near zero while delivering incredibly fast search speeds.	Consumes the most disk space among all methods because every password (even a 4-character one) is permanently expanded into a 32-character hexadecimal string.
Compressed Storage (zlib)	Maximizes hardware storage efficiency by significantly reducing the dataset's disk footprint (Compression Ratio < 1).	Suffers from massive latency and high memory overhead. The entire binary file must be decompressed and loaded into RAM before a single query can be executed, making it entirely impractical for real-time, user-facing query systems.

8. REFERENCES

1. R. Rivest, "The MD5 Message-Digest Algorithm," RFC 1321, April 1992.
<https://www.rfc-editor.org/rfc/rfc1321>
2. P. Deutsch, "DEFLATE Compressed Data Format Specification version 1.3," RFC 1951, May 1996. (Basis for zlib compression).
3. Richgel999, "miniz: Zlib compatible C/C++ data compression library (v3.0.2)," GitHub repository. <https://github.com/richgel999/miniz>
4. D. Miessler, "SecLists — Common-Credentials password lists," GitHub repository. <https://github.com/danielmiessler/SecLists/tree/master/Passwords/Common-Credentials>

Files utilized during preprocessing:

- 100k-most-used-passwords-NCSC.txt
- 10k-most-common.txt
- top-passwords-shortlist.txt
- xato-net-10-million-passwords-1000000.txt

5. Microsoft Corporation, "QueryPerformanceCounter function," Windows API Reference. <https://learn.microsoft.com/en-us/windows/win32/api/profileapi/nf-profileapi-queryperformancecounter>
6. Microsoft Corporation, "QueryPerformanceFrequency function," Windows API Reference. <https://learn.microsoft.com/en-us/windows/win32/api/profileapi/nf-profileapi-queryperformancefrequency>
7. Microsoft Corporation, "GetProcessMemoryInfo function (PSAPI)," Windows API Reference. (Used for tracking WorkingSetSize / RAM footprint). <https://learn.microsoft.com/en-us/windows/win32/api/psapi/nf-psapi-getprocessmemoryinfo>
8. Microsoft Corporation, "GetProcessTimes function," Windows API Reference. (Used for calculating User and Kernel mode CPU utilization). <https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-getprocesstimes>
9. Microsoft Corporation, "GetFileSizeEx and CreateFileA functions," Windows API Reference. (Used for high-speed file size querying without buffering).
10. MinGW-w64 Project, GCC (GNU Compiler Collection) for Windows. <https://www.mingw-w64.org/>